

Hadoop与Spark

HBase简介、部署与开发



HBase简介、部署与开发

◆ HBase简介

- ◆ HBase是一个高可靠、高性能、面向列、可伸缩的分布式数据库系统。利用HBase，可在廉价PC Server上搭建起大规模结构化数据管理集群
- ◆ HBase是一款借鉴Google Bigtable技术实现的开源软件
- ◆ 类似于Bigtable利用GFS作为其文件存储系统，HBase利用HDFS作为其文件存储系统
- ◆ Bigtable利用MapReduce来处理Bigtable中的海量数据，HBase则利用Hadoop MapReduce来处理HBase中的海量数据
- ◆ Bigtable利用 Chubby实现协同服务，HBase利用Zookeeper实现同样的功能

HBase简介、部署与开发

- ◆ HBase简介
 - ◆ 在Hadoop生态系统中，HBase为一款结构化数据管理工具
 - ◆ HDFS为HBase提供了高可靠性的底层存储支持，Hadoop MapReduce为HBase提供了高性能的计算能力，Zookeeper为HBase提供了稳定服务和Failover机制
 - ◆ Sqoop则为HBase提供了方便的RDBMS数据导入功能，使得从传统数据库向HBase迁移变的非常方便

HBase简介、部署与开发

- ◆ HBase的访问接口——丰富的访问接口
 - ◆ (1) Native Java API是最常用和高效的访问方式，适合于MapReduce Job并行处理HBase表格的数据
 - ◆ (2) HBase Shell，是HBase的命令行工具，适合于对HBase进行管理使用
 - ◆ (3) Thrift Gateway，利用Thrift序列化技术，支持C++、PHP、Python等多种语言，适合其它异构系统访问HBase表格数据
 - ◆ (4) REST Gateway，支持REST 风格的HTTP API访问HBase，解除了语言限制
 - ◆ (5) Pig，可以使用Pig Latin编程语言来操作HBase中的数据，需要编译成MapReduce Job来处理HBase表数据，适合做数据统计
 - ◆ (6) Hive，0.7.0以上版本的Hive支持HBase，使得用户可以使用类似SQL的查询语言来访问HBase

HBase简介、部署与开发

- ◆ HBase Use Cases



HBase简介、部署与开发

◆ HBase应用的特点

- ◆ Data volume
 - ◆ process petabytes of data in this distributed environment
- ◆ Application Types
 - ◆ we have a variable schema with slightly different rows
- ◆ Hardware Environment
 - ◆ works efficiently with a large number of nodes
- ◆ No requirement of relational features
 - ◆ do not need features like transaction, triggers, complex query, complex joins etc. then go for HBase
- ◆ Quick Access to data
 - ◆ while we require random and real-time access to our data, then we can use HBase



pause



HBase简介、部署与开发

- ◆ HBase的数据模型
 - ◆ Table: 一个Table就是HBase的一个表格，类似于关系数据库的表格，但是有所不同，请参考下文描述
 - ◆ Row Key: 行键，是Table的主键，Table中的记录按照Row Key排序
 - ◆ Column和Column Family (列簇或者列分组): Table在垂直方向由一个、或者多个Column Family组成
 - ◆ 一个Column Family可以由任意多个Column组成，即Column Family支持动态扩展，无需预先定义Column的数量以及类型
 - ◆ 所有Column均以二进制格式存储，用户需要自行进行类型转换
 - ◆ Timestamp: 时间戳，每次数据操作对应的时间戳，可以看作是数据的版本号(Version Number)

HBase简介、部署与开发

◆ HBase的数据模型 (实例)

◆ 这个表格有两个Column Family，分别是Contents和Anchor

- ◆ Contents保留了页面的内容，每个时间戳对应一个页面，可以保留该页面的不同历史版本

- ◆ Anchor则保存了指向这个页面(即引用该页面)的其它页面的锚点(即其它页面的超链接，指向本页面)的文本信息。

- ◆ 在这个实例里面，CNN的主页，被Sports Illustrated(cnnsi.com)和My Look(my.look.ca)两个页面所指向，所以每行记录有两列anchor:cnnsi.com和anchor: my.look.ca，它们隶属于同一个Column Family，即anchor

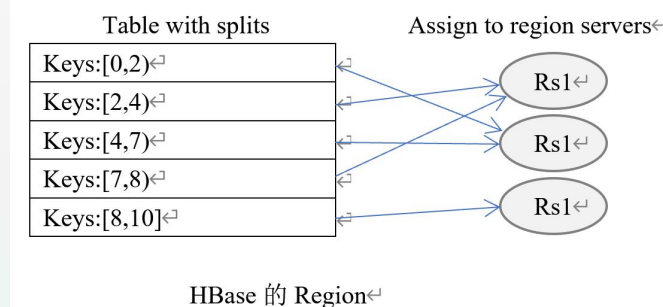
- ◆ 这个表格有两个row key，第一个row key保留了三个时间戳的contents列，一个时间戳的 anchor:cnnsi.com 列，以及另外一个时间戳的 anchor:my.look.ca列

Row key	Column family1		Column family2	
	timestamp	Contents 列	anchor:cnnsi.com 列	anchor:my.look.ca 列
com.cnn.www	t3	<html>.....	t9 "CNN"	t8 "CNN.com"
	t5	<html>.....		
	t6	<html>.....		
com.abc.www				

Hbase 的数据模型

HBase简介、部署与开发

- ◆ HBase的数据模型
 - ◆ Region
 - ◆ 当Table记录数不断增加而变大后，逐渐分裂成多个Split，称为Region
 - ◆ 每个Region由[startkey,endkey)表示，不同的Region被Master分配给相应的RegionServer(请参考下文描述)进行管理(存储)

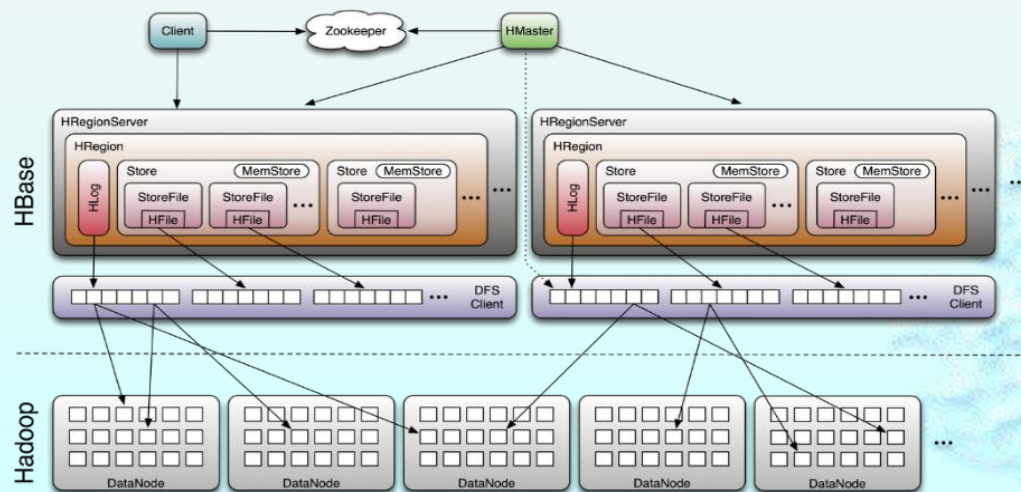


- ◆ HBase中有两张特殊的Table，“-ROOT-”和“.META.”
- ◆ (1) .META.: 记录了用户表的Region信息，.META.可以有多个region
- ◆ (2) -ROOT-: 记录了.META.表的Region信息，-ROOT-只有一个region
- ◆ Zookeeper中记录了-ROOT-表的位置信息(Location)
 - ◆ Client访问数据之前需要首先访问zookeeper，然后访问-ROOT-表，接着访问.META.表，进而找到用户数据的位置，最后去访问具体的数据

HBase简介、部署与开发

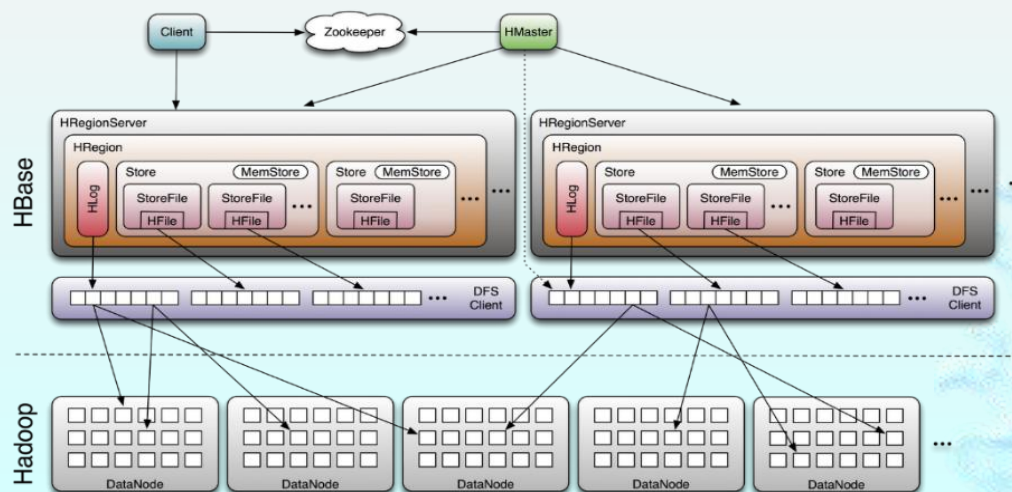
◆ HBase的系统架构

- ◆ 是一个典型的Master-Slave架构，包含HMaster和HRegionServer，以及Zookeeper
 - ◆ **Client**: HBase Client使用HBase的RPC机制与HMaster和HRegionServer进行通信
 - ◆ 对于管理类操作，Client通过RPC访问Hmaster
 - ◆ 对于数据读/写类操作，Client通过RPC访问HRegionServer
 - ◆ **Zookeeper**: Zookeeper Quorum存储-ROOT-表的地址和HMaster的地址
 - ◆ HRegionServer把自己以Ephemeral(短暂的)方式，注册到 Zookeeper中，使得HMaster可以随时感知到各个HRegionServer的健康状态
 - ◆ 引入Zookeeper，避免了HMaster的单点失败问题



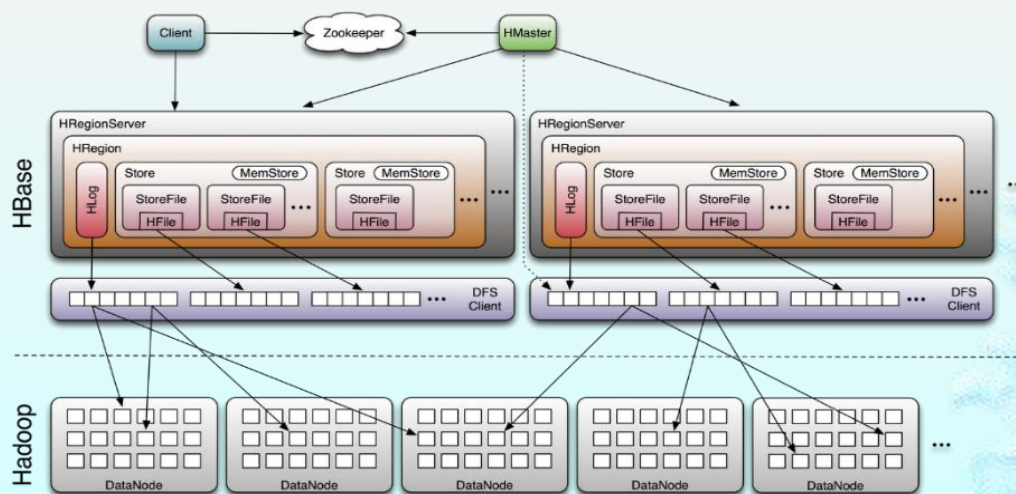
HBase简介、部署与开发

- ◆ HBase的系统架构
 - ◆ **HMaster**: HMaster负责Table和Region的管理工作，具体包括：
 - ◆ (1) 管理用户对Table的增、删、改、查操作；
 - ◆ (2) 管理HRegionServer的负载均衡，调整Region分布；
 - ◆ (3) 在Region分裂(Split)后，负责分配新的Region；
 - ◆ (4) 在HRegionServer停机后，负责失效HRegionServer 上的Region的迁移
 - ◆ HBase可以启动多个HMaster，通过Zookeeper的Master Election机制，保证总有一个Active Master运行，所以HMaster没有单点失败问题



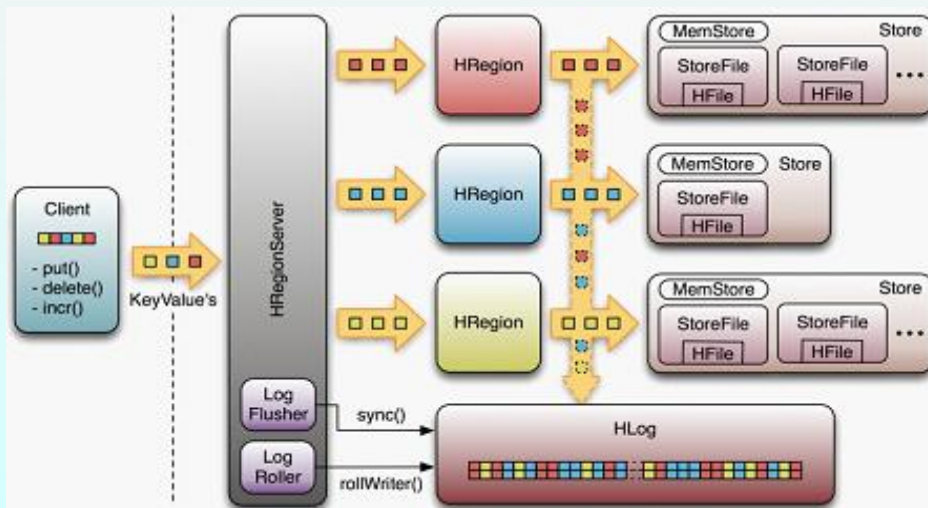
HBase简介、部署与开发

- ◆ HBase的系统架构
 - ◆ **HRegionServer**: HRegionServer主要负责响应用户I/O请求，向HDFS文件系统读/写数据，是HBase中最核心的模块。
 - ◆ 每个 Region Server 大约可以管理 1000 个 Regions



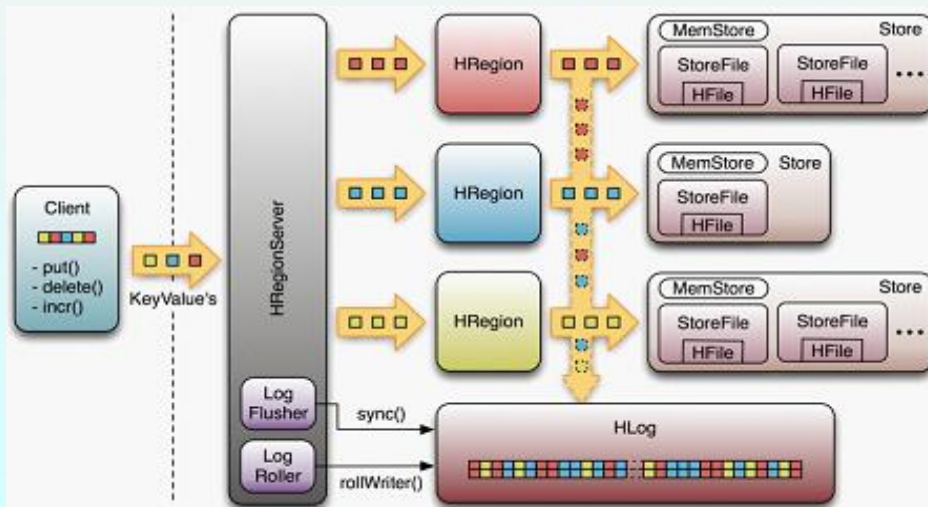
HBase简介、部署与开发

- ◆ HBase的系统架构
 - ◆ **HRegionServer的内部结构**
 - ◆ HRegionServer内部管理一系列HRegion对象，每个HRegion对应了Table中的一个 Region
 - ◆ HRegion由多个HStore组成，每个HStore对应了Table中的一个 Column Family的存储
 - ◆ 可以看出每个Column Family是一个集中的存储单元，因此最好将具备共同I/O特性的Column放在一个Column Family中，可提高IO效率



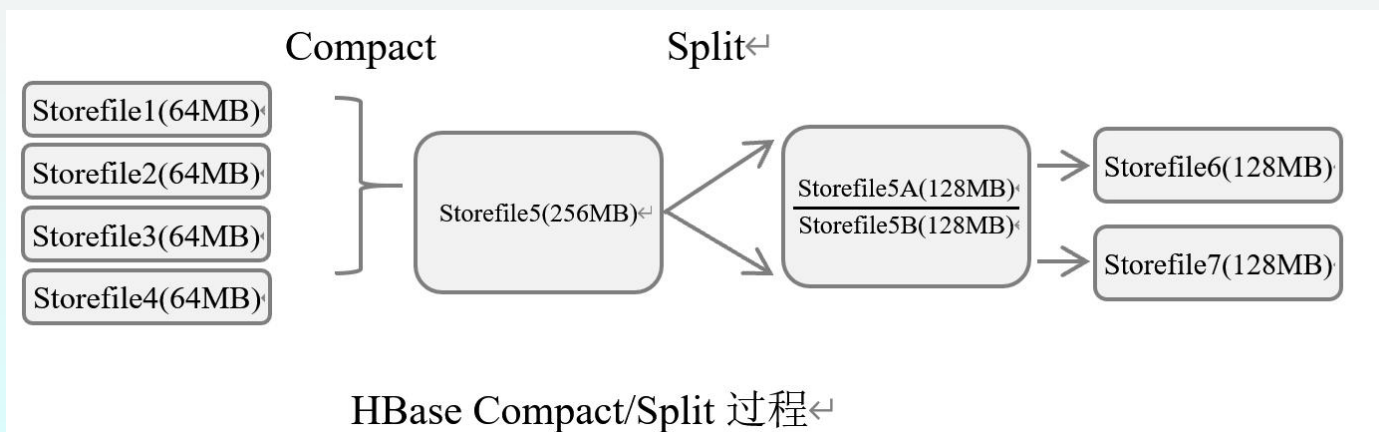
HBase简介、部署与开发

- ◆ HBase的系统架构
 - ◆ HRegionServer的内部结构
 - ◆ **HStore**存储是HBase存储的核心模块，由两部分组成，分别是MemStore和一系列StoreFile
 - ◆ MemStore是Sorted Memory Buffer，用户写入的数据首先会放入MemStore，当MemStore满了以后，写入(Flush)一个StoreFile(底层实现是HFile)
 - ◆ 当StoreFile文件数量增长到一定阈值，会触发Compact合并操作，将多个StoreFile合并成一个StoreFile，合并过程中会进行版本合并和数据删除
 - ◆ HBase只能增加数据，所有的更新和删除操作都是在后续的Compact过程中进行的
 - ◆ 这使得用户的写操作只要进入内存中就可以立即返回，保证了HBase I/O的高性能



HBase简介、部署与开发

- ◆ HBase的系统架构
 - ◆ HRegionServer的内部结构
 - ◆ 当StoreFile Compact后，会逐步形成越来越大的StoreFile，当单个StoreFile大小超过一定阈值后，会触发Split操作，把当前 Region Split成2个Region
 - ◆ 父Region会下线，新Split出的2个子Region会被HMaster分配到相应的HRegionServer 上，使得原先1个Region的压力得以分流到2个Region上



HBase简介、部署与开发

- ◆ HBase的系统架构

- ◆ Hlog

- ◆ 在分布式系统环境中，无法避免系统出错或者宕机
 - ◆ 一旦HRegionServer意外退出，MemStore中的内存数据将会丢失，需要引入Hlog
 - ◆ 每个HRegionServer都有一个HLog对象，HLog是一个实现Write Ahead Log的类
 - ◆ 在每次用户操作写入MemStore时，首先写一份数据到HLog文件中；HLog文件定期删除旧的日志(已持久化到StoreFile中的数据)
 - ◆ 当HRegionServer意外终止后，HMaster会通过Zookeeper感知到这个情况，HMaster首先会处理遗留的 HLog文件，将其中不同Region的Log数据进行拆分，分别放到相应Region的目录下，然后再将失效的Region重新分配
 - ◆ 领取到这些Region的HRegionServer在装载Region的过程中，会发现历史HLog需要处理，因此会重放HLog(Applying Log Records)中的数据到MemStore中，然后Flush到StoreFile中，完成数据恢复

HBase简介、部署与开发

- ◆ HBase存储格式
 - ◆ HBase的所有数据文件都存储在Hadoop HDFS文件系统上，包括两种文件类型。
 - ◆ (1) HFile, HFile是Hadoop的二进制格式文件，实现HBase中KeyValue数据的存储
 - ◆ StoreFile是对HFile做了轻量级包装，即StoreFile底层就是HFile
 - ◆ (2) HLog File, HBase中实现WAL(Write Ahead Log) 的存储格式，本质上是Hadoop的Sequence File

HBase简介、部署与开发

◆ HBase存储格式

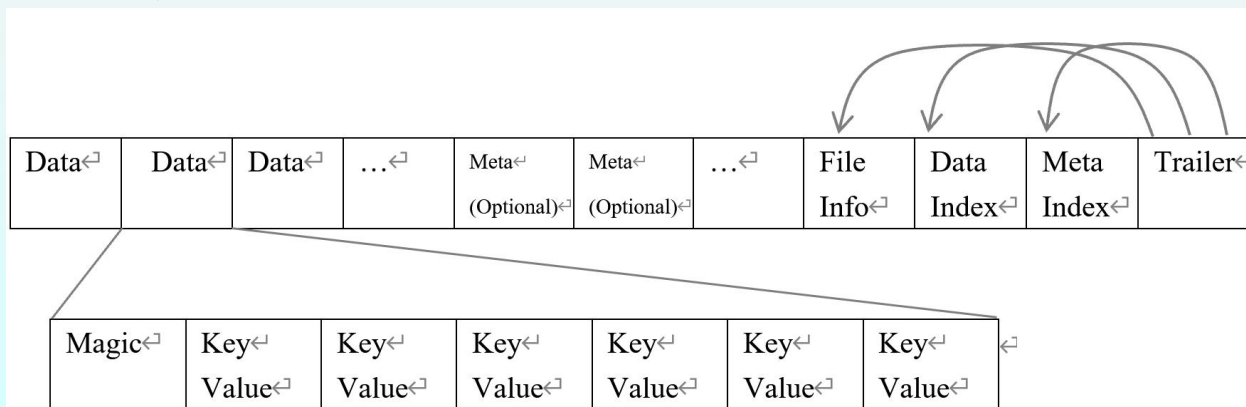
◆ HFile结构

◆ HFile文件是不定长的，长度固定的只有其中的两块：Trailer和FileInfo。Trailer 中有指针指向其它数据块的起始点。FileInfo中记录了文件的一些元信息，比如AVG_KEY_LEN、AVG_VALUE_LEN、LAST_KEY、COMPARATOR、MAX_SEQ_ID_KEY等。Data Index和Meta Index块记录了每个Data块和Meta块的起始点。形成了一种自描述的文件结构。

◆ Data块是HBase I/O的基本单元

◆ 为了提高效率，HRegionServer中实现了基于LRU的Block Cache机制

◆ 每个Data块的大小可以在创建一个Table的时候通过参数指定，较大的Block有利于顺序扫描，较小的Block利于随机查询



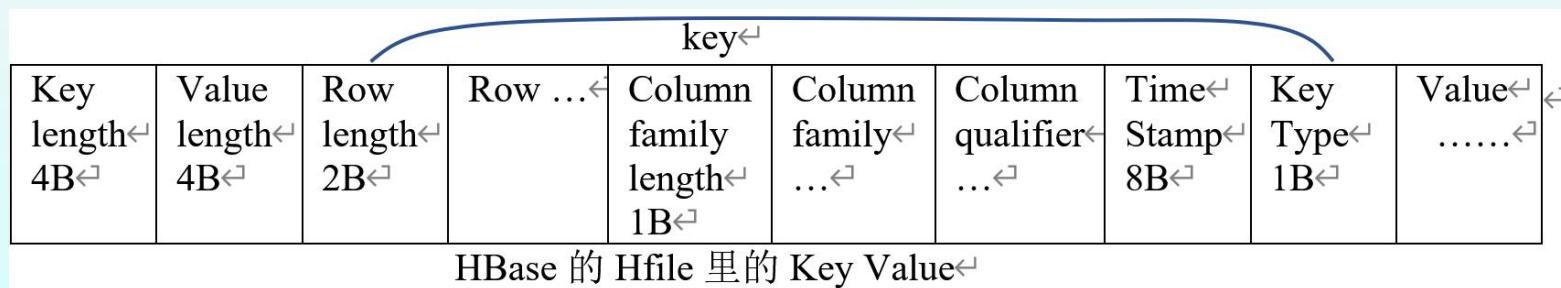
HBase 的 Hfile 存储格式

HBase简介、部署与开发

◆ HBase存储格式

◆ HFile结构

- ◆ 每个Data块除了开头的Magic信息以外，就是由一个个<Key,Value>对拼接而成的。Magic内容是一些随机数字，目的是防止数据损坏。
- ◆ HFile里面的每个KeyValue对是一个简单的Byte数组，但是这个Byte数组里面包含了很多项，并且有固定的结构。
- ◆ 开始是两个固定长度的数值，分别表示Key的长度和Value的长度。紧接着是Key部分，开始是固定长度的数值，表示RowKey 的长度，紧接着是RowKey，然后是固定长度的数值，表示Column Family的长度，然后是Column Family(如图1中的anchor)，接着是Column Qualifier(如图1中的cnnsi.com或者my.look.ca)，然后是两个固定长度的数值，表示Time Stamp和Key Type(Put/Delete)
- ◆ Value部分没有这么复杂的结构，就是纯粹的二进制数据



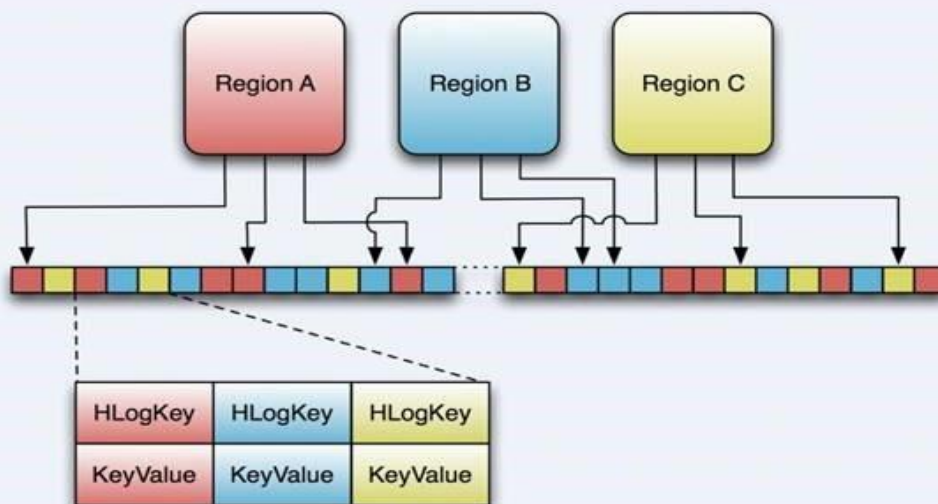
HBase简介、部署与开发

- ◆ HBase存储格式

- ◆ HLog文件

- ◆ 就是一个普通的Hadoop Sequence File。Sequence File 的Key是HLogKey对象，HLogKey中记录了写入数据的归属信息，除了Table和Region名字外，同时还包括Sequence Number和Timestamp

- ◆ Timestamp是“写入时间”，Sequence Number的起始值为0，或者是最近一次存入文件系统的Sequence Number。HLog Sequence File的Value是HBase的Key Value对象，即对应HFile中的Key和Value



HBase

◆ Zookeeper

- ◆ (1) 维护管理了Hbase相关元数据信息
 - ◆ ZK中meta-region-server节点维护hbase的meta表所在的节点信息
 - ◆ Hbase集群中会维护着一个meta表，这个表存储着Hbase集群所有region的相关信息，包括region所在RegionServer节点，region的rowkey范围，以及region所在表名、列值等信息
- ◆ (2) 实现Hmaster节点的高可用管理
- ◆ (3) 对集群所有RegionServer状态监控，宕机后会通知Hmaster；每个RS启动时都会会在ZK上注册，然后master会去监控这个znode
 - ◆ 在其他Region Server恢复相关Region，需要将已有HLog按照Region进行切分，并且根据需要进行重放
- ◆ (4) 分布式一致锁的提供，保证数据写入的事务性，表的删除，新建，更新需要维护全局锁

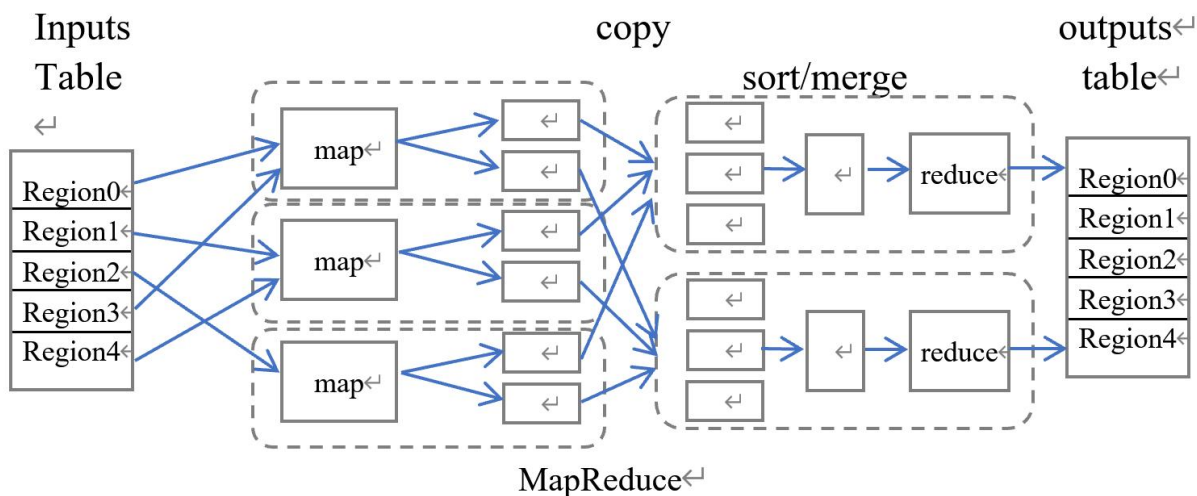
pause



HBase简介、部署与开发

◆ MapReduce on HBase

- ◆ 可以在HBase系统上运行MapReduce Job，实现数据的批处理
- ◆ HBase Table和Region的关系，类似于HDFS File和Block的关系
- ◆ HBase提供了配套的TableInputFormat和TableOutputFormat API，可以方便的将HBase Table作为Hadoop MapReduce的Source和Sink
- ◆ 对于MapReduce Job应用开发人员来说，不需要关注HBase系统的细节



MapReduce on HBase 原理

pause



HBase简介、部署与开发

- ◆ HBase安装、配置与运行
- ◆ 从<http://www-eu.apache.org/dist/hbase/stable/>下载HBase安装包
- ◆ 解压缩，对目录进行重新命名

```
cd /opt/linuxsir  
tar xzvf hbase-1.2.5-bin.tar.gz  
ls hbase-1.2.5  
  
mv hbase-1.2.5 /opt/linuxsir/hbase
```

- ◆ HBase可以以Standalone、伪分布、**分布式**等模式进行部署，这里只介绍分布式的部署模式

HBase简介、部署与开发

- ◆ **HBase安装、配置与运行**
- ◆ 修改/opt/linuxsir/HBase/conf目录下的HBase-env.sh脚本文件。设置JAVA_HOME的环境变量，以便启动HBase

```
export JAVA_HOME=/opt/linuxsir/java/jdk
export HBASE_CLASSPATH=/opt/linuxsir/hadoop/etc/hadoop

export HBASE_MANAGES_ZK=true
```

HBase简介、部署与开发

- ◆ **HBase安装、配置与运行**
- ◆ 修改当前用户目录下的~/.bashrc文件，并使之生效

```
echo "">> ~/.bashrc
echo "export HBASE_HOME=/opt/linuxsir/HBase">> ~/.bashrc
echo "export PATH=\$HBASE_HOME/bin:\$PATH">> ~/.bashrc
echo "export CLASSPATH=\$CLASSPATH:/opt/linuxsir/hbase/lib/*">> ~/.bashrc

cat ~/.bashrc
source ~/.bashrc
```

HBase简介、部署与开发

- ◆ HBase安装、配置与运行
- ◆ 执行如下命令，建立一些目录，包括hbase/zookeeper等目录

```
mkdir /opt/linuxsir/hbase/tmp
```

```
ls /opt/linuxsir/hbase/zookeeper
```

\\没有该目录就创建

```
mkdir /opt/linuxsir/hbase/zookeeper
```

HBase简介、部署与开发

- ◆ HBase安装、配置与运行
- ◆ 编辑HBase配置文件，即/opt/linuxsir/hbase/conf目录下的hbase-site.xml，内容如下

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
  <property>
    <name>HBase.rootdir</name><!-- 用户无需手动创建HBase的data目录，HBase启动的时候会自动创建-->
    <value>hdfs://hd-master:9000/hbase</value>
  </property>

  <property>
    <name>HBase.zookeeper.property.dataDir</name>
    <value>/opt/linuxsir/hbase/zookeeper</value>
  </property>

  <property>
    <name>HBase.cluster.distributed</name>
    <value>true</value>
  </property>
```

HBase简介、部署与开发

- ◆ **HBase安装、配置与运行**
- ◆ 编辑HBase配置文件，即/opt/linuxsir/hbase/conf目录下的hbase-site.xml，内容如下

```
<property>
  <name>HBase.zookeeper.quorum</name>
  <value>hd-master,hd-slave1,hd-slave2</value>
</property>

<property>
  <name>HBase.zookeeper.property.dataDir</name>
  <value>/opt/linuxsir/hbase/zookeeper</value>
</property>

<property>
  <name>HBase.zookeeper.property.clientPort</name>
  <value>12181</value>
</property>
</configuration>
```

HBase简介、部署与开发

◆ HBase安装、配置与运行

◆ 在一个分布式部署模式下，HBase集群包含若干节点，运行主HMaster进程、Backup HMaster进程、HRegionServer进程、以及Zookeeper进程等。

◆ 在这里，我们采用的如下部署规划，注意Zookeeper的部署，已经在HBase-site.xml里面设置。

- ◆ (1) HMaster进程，运行在hd-master, hd-slave1 (backup)上
- ◆ (2) ZooKeeper进程，运行在hd-master, hd-slave1,hd-slave2上
- ◆ (2) RegionServer进程，运行在hd-slave1,hd-slave2上

HBase简介、部署与开发

- ◆ HBase安装、配置与运行
 - ◆ 编辑/opt/linuxsir/HBase/conf目录下的regionservers文件，内容如下

```
hd-slave1  
hd-slave2
```

HBase简介、部署与开发

- ◆ HBase安装、配置与运行

- ◆ 编辑/opt/linuxsir/HBase/conf目录下的`backup-masters`文件，内容如下

```
hd-slave1
```

- ◆ 注意，这里我们没有看到HMaster的部署位置的配置
- ◆ 当我们在hd-master上启动HBase，那么hd-master节点将运行HMaster

HBase简介、部署与开发

- ◆ HBase安装、配置与运行
 - ◆ 启动HBase并且测试
 - ◆ 从hd-master拷贝/opt/linuxsir/HBase目录、~/.bashrc文件到hd-slave1和hd-slave2。并且在三个节点上，使得~/.bashrc文件生效

```
clear
scp -r /opt/linuxsir/hbase hd-slave1:/opt/linuxsir
scp -r /opt/linuxsir/hbase hd-slave2:/opt/linuxsir

scp ~/.bashrc hd-slave1:~/.bashrc
scp ~/.bashrc hd-slave2:~/.bashrc

source ~/.bashrc
ssh root@192.168.31.130 source ~/.bashrc
ssh root@192.168.31.131 source ~/.bashrc
```

HBase简介、部署与开发

- ◆ HBase安装、配置与运行
 - ◆ 启动HBase之前，需要事先启动Hadoop
 - ◆ 首先，清空日志目录，以便启动出错时，方便查看最新出错信息

```
clear  
cd /opt/linuxsir/hadoop/sbin  
./start-dfs.sh  
./start-yarn.sh
```

```
rm -rf /opt/linuxsir/HBase/logs/*.*  
ls /opt/linuxsir/HBase/logs
```

```
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hbase/logs/*.*  
ssh root@192.168.31.130 ls /opt/linuxsir/hbase/logs
```

```
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hbase/logs/*.*  
ssh root@192.168.31.131 ls /opt/linuxsir/hbase/logs
```

HBase简介、部署与开发

- ◆ **HBase安装、配置与运行**
 - ◆ 在hd-master节点上启动HBase，并且检查相关进程是否都正常启动
 - ◆ 三台虚拟机上运行的进程为
 - ◆ hd-master有HMaster和HQuorumPeer
 - ◆ hd-slave1上有HMaster、HRegionServer和HQuorumPeer
 - ◆ hd-slave2上有HRegionServer和HQuorumPeer
 - ◆ 启动过程中，首先内置的ZooKeeper启动，接着HMaster启动，然后HRegionServer(多个节点)启动，最后是Backup HMaster启动
 - ◆ 需要注意的是，各个节点还应该启动了HDFS和YARN的进程

```
cd /opt/linuxsir/hbase  
./bin/start-hbase.sh
```

```
jps  
ssh root@192.168.31.130 jps  
ssh root@192.168.31.131 jps
```

HBase简介、部署与开发

- ◆ HBase安装、配置与运行
 - ◆ 应该看到如下进程列表，进程号有差别

\\在hd-master上的进程(进程号可能不同)

20355 Jps

20071 HQuorumPeer

20137 HMaster

\\在hd-slave1上的进程

15930 HRegionServe

16194 Jps

15838 HQuorumPeer

16010 HMaster

HBase 层↵	HQuorumPeer ↵	HQuorumPeer↵	HQuorumPeer↵	...↵
	HMaster↵ ↵	HMaster↵ HRegionServer↵	↵ HRegionServer↵	
YARN 层↵	ResourceManager↵	NodeManager↵	NodeManager↵	...↵
HDFS 层↵	NameNode↵ Secondary NameNode↵	DataNode↵	DataNode↵	...↵
Hardware 各个节点↵	hd-master 节点↵ 192.168.31.129↵	hd-slave1 节点↵ 192.168.31.130↵	hd-slave2 节点↵ 192.168.31.131↵	...↵

HBase 进程↵

\\在hd-slave2上的进程

13901 Jps

13639 HQuorumPeer

13737 HRegionServer

HBase简介、部署与开发

- ◆ **HBase安装、配置与运行**

- ◆ 检查hdfs://hd-master:9000/HBase分布式文件系统目录是否存在(HBase自动创建)

```
cd /opt/linuxsir/hadoop/bin  
./hdfs dfs -ls /hbase
```

- ◆ 利用HBase Web 管理界面查看HBase状态

```
http://192.168.31.129:16010  
\\可以查看HBase的各个端口
```

pause



HBase简介、部署与开发

- ◆ HBase安装、配置与运行
- ◆ 使用Shell程序

```
\\start up hbase shell  
cd /opt/linuxsir/hbase  
./bin/hbase shell
```

HBase简介、部署与开发

- ◆ HBase安装、配置与运行
- ◆ 在HBase Shell里面，进行建表，增删改查操作

```
\\Create a table  
create 'test', 'cf'
```

```
\\显示Table信息  
list 'test'
```

```
\\插入数据  
put 'test', 'row7', 'cf:a', 'value7a'  
put 'test', 'row7', 'cf:b', 'value7b'  
put 'test', 'row7', 'cf:c', 'value7c'
```

```
put 'test', 'row8', 'cf:b', 'value8b'  
put 'test', 'row9', 'cf:c', 'value9c'
```

\\cf为Column Family的名称，a、b、c等为字段名称

HBase简介、部署与开发

- ◆ HBase安装、配置与运行
- ◆ 在HBase Shell里面，进行建表，增删改查操作

\\扫描显示整个Table的数据
`scan 'test'`

\\提取一行数据
`get 'test', 'row7'`

\\Disable一个表格
`disable 'test'`
`enable 'test'`

\\删除表格
`drop 'test'`

\\退出HBase Shell
`quit`

HBase简介、部署与开发

- ◆ HBase安装、配置与运行
- ◆ 停止HBase
- ◆ 停止Hadoop

```
cd /opt/linuxsir/hbase  
./bin/stop-hbase.sh
```

```
cd /opt/linuxsir/hadoop/sbin  
./stop-yarn.sh  
./stop-dfs.sh
```

pause



HBase简介、部署与开发

- ◆ HBase Java实例分析
- ◆ 首先, 导入必要的类

```
package com.pai.HBase;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.HBase.HBaseConfiguration;
import org.apache.hadoop.HBase.HColumnDescriptor;
import org.apache.hadoop.HBase.HTableDescriptor;
import org.apache.hadoop.HBase.TableName;
import org.apache.hadoop.HBase.client.*;
import org.apache.hadoop.HBase.util.Bytes;
import org.apache.log4j.PropertyConfigurator;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.io.IOException;
import java.util.UUID;
```

HBase简介、部署与开发

- ◆ **HBase Java实例分析**
- ◆ 创建App类，为其编写main函数
- ◆ 新建Configuration类的对象conf，设置HBase.zookeeper.quorum属性为HBase的内置Zookeeper的IP地址，并设置HBase.zookeeper.property.clientPort属性为HBase的内置Zookeeper的端口
- ◆ 利用conf对象，创建到HBase的连接。对于HBase客户端来讲，只需要指定Zookeeper服务器在哪里，端口是什么即可

```
public class App {  
  
    private static final Logger logger = LoggerFactory.getLogger(App.class);  
  
    public static void main(String[] args) throws IOException {  
  
        Configuration conf = HBaseConfiguration.create();  
        // Option 1 (recommended): Place HBase-site.xml on src/main/resources  
        // Option 2: Configure the HBase connection properties here:  
        // - Zookeeper addresses  
        conf.set("HBase.zookeeper.property.clientPort", "12181");  
        conf.set("HBase.zookeeper.quorum", "192.168.31.129");  
  
        // Create a connection (you can use a try with resources block)  
        Connection connection = ConnectionFactory.createConnection(conf);  
    }  
}
```

HBase简介、部署与开发

- ◆ HBase Java实例分析
- ◆ 如下代码创建表格
- ◆ 需要指定表格名称和Column Family的名称，可以有多个Column Family

```
// Creating a table with a random name
Admin admin = connection.getAdmin();
UUID uuid = UUID.randomUUID();
TableName tableName = TableName.valueOf(uuid.toString());
HTableDescriptor tableDescriptor = new HTableDescriptor(tableName);
HColumnDescriptor cf = new HColumnDescriptor(Bytes.toBytes("cf"));
tableDescriptor.addFamily(cf);
logger.info("Creating table {}", tableName);
admin.createTable(tableDescriptor);
```

HBase简介、部署与开发

- ◆ HBase Java实例分析
- ◆ 接着插入一行数据，然后再利用for循环插入若干行数据

```
// Adding a row and more rows
Table table = connection.getTable(tableName);

logger.info("Adding a row");
Put put = new Put(Bytes.toBytes("0001"));
put.addColumn(Bytes.toBytes("cf"), Bytes.toBytes("name"), Bytes.toBytes("Javier"));
table.put(put);

for (int i = 2; i < 10; i++) {
    Put p = new Put(Bytes.toBytes("000" + i));
    p.addColumn(Bytes.toBytes("cf"), Bytes.toBytes("name"), Bytes.toBytes("Javier-" +
i));
    table.put(p);
}
```

HBase简介、部署与开发

- ◆ HBase Java实例分析
- ◆ 读取一行数据，需要指定key

```
// Reading a row
logger.info("Reading a row");
Get get = new Get(Bytes.toBytes("0001"));
Result r = table.get(get);
byte[] value = r.getValue(Bytes.toBytes("cf"), Bytes.toBytes("name"));
logger.info("Result: Row: {} Value: {}", r, Bytes.toString(value));
```

HBase简介、部署与开发

- ◆ HBase Java实例分析
- ◆ 读取所有数据

```
// Full scan
logger.info("Full scan");
Scan scan = new Scan();
ResultScanner scanner = table.getScanner(scan);
for (Result res : scanner) {
    logger.info("Row: {}", res);
}
scanner.close();
```

HBase简介、部署与开发

- ◆ HBase Java实例分析
- ◆ 根据条件，**读取符合条件的数据**

```
// Restricted Scan
logger.info("Restricted Scan");
Scan scan2 = new Scan();
scan2.addColumn(Bytes.toBytes("cf"), Bytes.toBytes("name")).setStartRow(Bytes.toBytes("0003"))
    .setStopRow(Bytes.toBytes("0005"));
ResultScanner scanner2 = table.getScanner(scan2);
for (Result res2 : scanner2) {
    System.out.println(res2);
    byte[] value2 = res2.getValue(Bytes.toBytes("cf"), Bytes.toBytes("name"));
    System.out.println("Value: " + Bytes.toString(value2));
}
scanner2.close();
```

HBase简介、部署与开发

- ◆ HBase Java实例分析
- ◆ 关闭表格, Disable表格和删除表格
- ◆ 最后关闭连接, 结束程序

```
// Close the table
table.close();

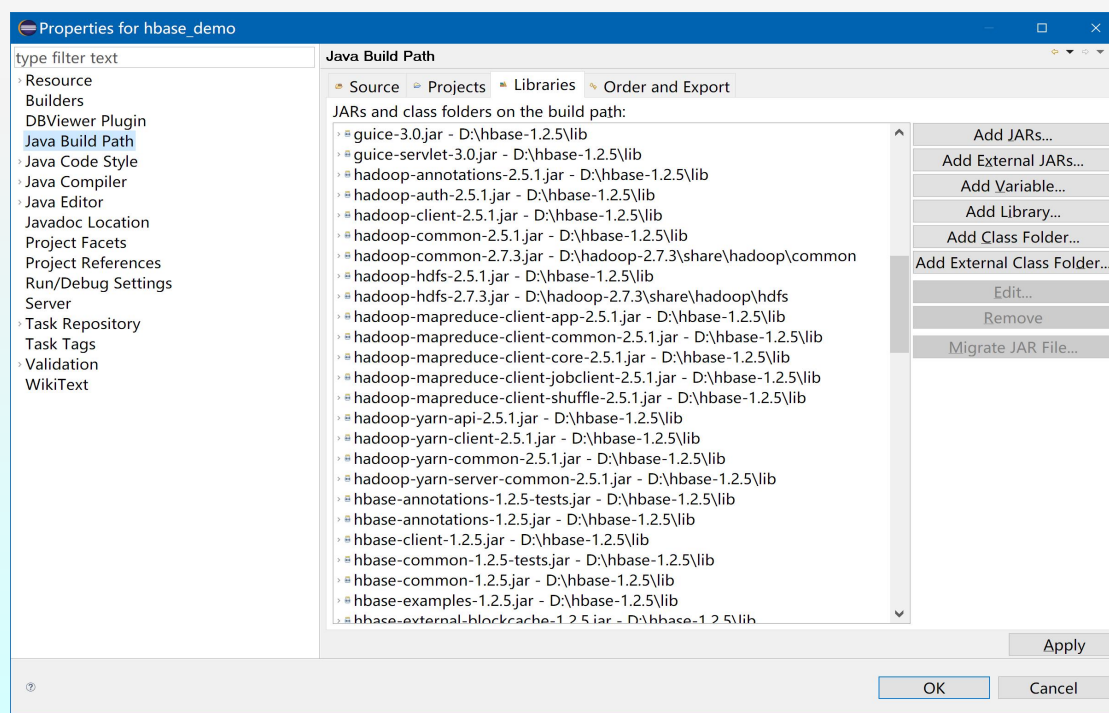
// Delete the table
logger.info("Disabling table {}", tableName);
admin.disableTable(tableName);
logger.info("Deleting table {}", tableName);
admin.deleteTable(tableName);

// Close the connection
connection.close();

System.exit(0);
}
```

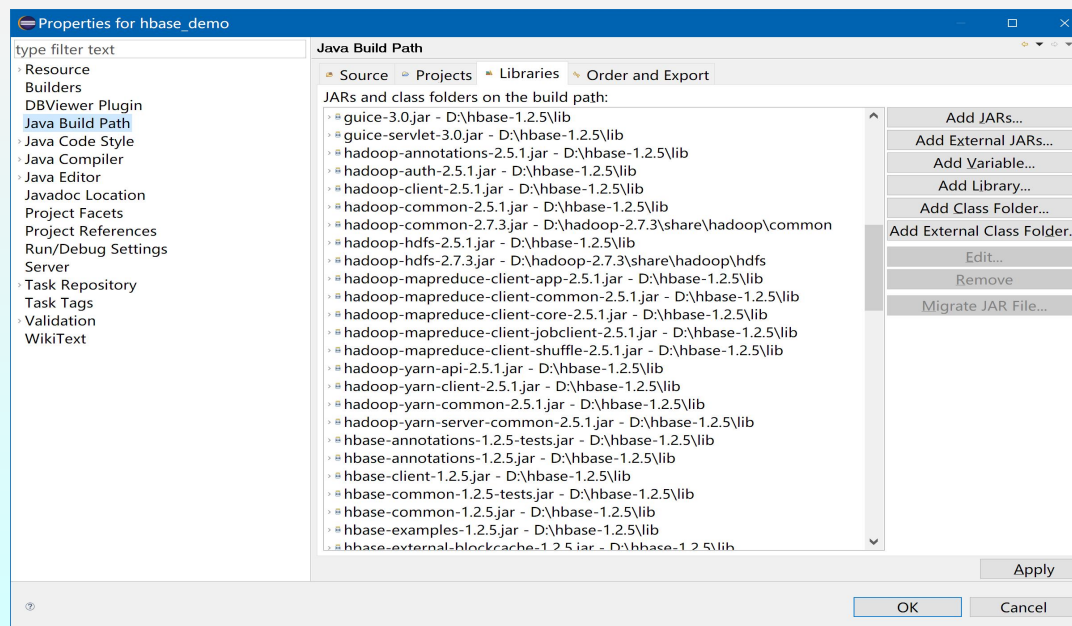
HBase简介、部署与开发

- ◆ HBase Java实例分析
 - ◆ 需要注意的是，为了使得程序编译通过，能够运行，需要把HBase安装包解压缩到d:\hbase-1.2.5目录下，并且把解压缩目录的lib子目录下的jar包加入Eclipse项目的Build Path
 - ◆ 在Eclipse的Java项目上，按右键，选择Properties，打开对话框，选择Java Build Path，显示如下对话框



HBase简介、部署与开发

- ◆ HBase Java实例分析
 - ◆ 把d:\hadoop-2.7.3\share\hadoop\common下的hadoop-common-2.7.3.jar
 - ◆ 以及d:\hadoop-2.7.3\share\hadoop\common下hadoop-hdfs-2.7.3.jar,
 - ◆ 还有d:\hbase-1.2.5\lib目录下的所有jar包,
 - ◆ 通过Add External Jars按钮, 加入项目的Build Path



HBase简介、部署与开发

- ◆ HBase Java实例分析
 - ◆ 为了运行或者调试上述代码，记得启动Hbase
 - ◆ 当然首先需要启动HDFS和YARN

```
rm -rf /opt/linuxsir/hadoop/logs/*.*  
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hadoop/logs/*.*  
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hadoop/logs/*.*
```

```
clear  
cd /opt/linuxsir/hadoop/sbin  
./start-dfs.sh  
./start-yarn.sh
```

```
cd /opt/linuxsir/hbase  
./bin/start-hbase.sh
```

```
clear  
jps  
ssh root@192.168.31.130 jps  
ssh root@192.168.31.131 jps
```

HBase简介、部署与开发

- ◆ HBase Java实例分析
 - ◆ 关闭HBase用如下命令

```
cd /opt/linuxsir/hbase  
./bin/stop-hbase.sh
```

```
cd /opt/linuxsir/hadoop/sbin  
./stop-yarn.sh  
./stop-dfs.sh
```

```
jps  
ssh root@192.168.31.130 jps  
ssh root@192.168.31.131 jps
```

pause



HBase简介、部署与开发

- ◆ 问题1：端口被占用问题解决
- ◆ 2181被占用，于是我们改了/opt/linuxsir/HBase/conf/HBase-site.xml里面的HBase.zookeeper.property.clientPort为12181

```
<property>  
  <name>hbase.zookeeper.property.clientPort</name>  
  <value>12181</value>  
</property>
```

- ◆ 停止HBase，从hd-master复制配置文件到hd-slave1、hd-slave2两个节点，然后再启动HBase

```
scp /opt/linuxsir/hbase/conf/hbase-site.xml hd-slave1:/opt/linuxsir/hbase/conf  
scp /opt/linuxsir/hbase/conf/hbase-site.xml hd-slave2:/opt/linuxsir/hbase/conf
```

HBase简介、部署与开发

- ◆ 问题2: 提示 “QuorumCnxManager: Cannot open channel to 2 at election address hd-slave2/192.168.31.131:3888”
- ◆ 在hd-master上运行如下命令, 同步三个节点的hosts文件

```
clear  
scp /etc/hosts root@192.168.31.130:/etc/hosts  
scp /etc/hosts root@192.168.31.131:/etc/hosts
```

HBase简介、部署与开发

- ◆ 问题3: 提示“zookeeper.MetaTableLocator: Failed verification of HBase:meta”。
- ◆ 解决办法如下，启动HBase之前执行如下命令，重建/opt/linuxsir/HBase/zookeeper目录
- ◆ 停止HBase，从hd-master复制配置文件到hd-slave1、hd-slave2两个节点，然后再启动HBase

```
rm -rf /opt/linuxsir/hbase/zookeeper  
mkdir /opt/linuxsir/hbase /zookeeper
```

```
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hbase /zookeeper  
ssh root@192.168.31.130 mkdir /opt/linuxsir/hbase /zookeeper
```

```
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hbase /zookeeper  
ssh root@192.168.31.131 mkdir /opt/linuxsir/hbase /zookeeper
```

HBase简介、部署与开发

- ◆ 问题4: 提示“util.FSUtils: Waiting for dfs to exit safe mode...”。
- ◆ 解决办法如下，启动HBase之前执行如下命令，强制NameNode退出SafeMode模式

```
cd /opt/linuxsir/hadoop/bin  
hdfs dfsadmin -safemode leave
```

pause



HBase简介、部署与开发

- ◆ 调研报告
 - ◆ HBase Secondary Index Support?



The screenshot shows a Google search interface with the query "HBase secondary index coprocessor example". The search results list two articles. The first article is from "www.programmersought.com" and is titled "Hbase coprocessor and secondary index - Programmer Sought". The second article is from "www.agilelab.it" and is titled "Secondary Indexing on HBase, a.k.a. NoSQL is good but the ...".

Google

HBase secondary index coprocessor example

www.programmersought.com › article ▼ 翻译此页

Hbase coprocessor and secondary index - Programmer Sought

Taking the HBase 0.92 version as an example, it provides three observer interfaces: ○
RegionObserver: Provides client data manipulation event hooks: Get, Put, ...

www.agilelab.it › secondary-indexing-on-hba... ▼ 翻译此页

Secondary Indexing on HBase, a.k.a. NoSQL is good but the ...

2019年7月12日 - Secondary Indexing on HBase, a.k.a. NoSQL is good but the world is not key-value ... Don't panic, let's make an example ... "Secondary Indexing" suggests users to develop a custom "Coprocessor" that deserializes our "PUT" ...

pause

